

Building an async/await task library in C++20

REDPOINT

Welcome!

- I'm June (She/Her)
- I integrate Epic Online Services into Unreal Engine
- EOS plugin has a lot of asynchronous code (most EOS SDK calls are async)
- Chaining lots of sequential async calls is painful with callbacks
- I wrote an async task library with C++20 coroutines to make things more maintainable



REDPOINT™

Agenda

- Callbacks vs `co_await`
- C++ coroutines spec and task library design
- Building out the task library
 - Lifetime control
 - Imposed cancellation
 - Interacting with synchronous code
 - Requested cancellation
 - Async lambdas

REDPOINT

- Async lambdas if we get time
- *Forewarning:* I likely have more slides and code examples than we have time to cover, this topic gets very deep
- There are more concepts like thread policies that didn't even make this list
- I am going to try to cover as much as possible in the time we have
- If we run out of time, all of the slides and all of the examples are online, there will be a link at the end of the presentation

Callbacks vs `co_await`

REDPOINT

Callbacks are hard to maintain

1. Function returning on a different frame
 2. Pass a callback in, which gets called when it's complete
- Hard to statically guarantee that the function always calls the callback
 - Lambda callbacks get deeply nested after even a few asynchronous calls

```
MultiOperation(FUnresolvedFriendEntry, TResolvedFriendEntry)::RunBatched(
    EntryType.Value,
    EOS_CONNECT_QUERYEXTERNALACCOUNTMAPPINGS_MAX_ACCOUNT_IDS,
    [weakThis = GetWeakThis(this), EntryType, EOSUser]{
        TArray(FUnresolvedFriendEntry) Batch;
        const std::function<void(TArray(FResolvedFriendEntry) OutBatch)> &OnDone -> bool {
            if (auto This = PinWeakThis(weakThis))
            {
                EOS_Connect_QueryExternalAccountMappingsOptions Opts = {};
                Opts.ApiVersion = EOS_CONNECT_QUERYEXTERNALACCOUNTMAPPINGS_API_LATEST;
                Opts.AccountIdType = EntryType.Key;
                Opts.LocalUserId = EOSUser->GetProductId();
                EOSString_Connect_ExternalAccountId::AllocateToCharListViaAccessor(FUnresolvedFriendEntry{
                    Batch,
                });
                [[const FUnresolvedFriendEntry Entry] {
                    return Entry->ExternalInfo.AccountId;
                },
                Opts.ExternalAccountIdCount,
                Opts.ExternalAccountIds);
            }
        }

    EOSOperationLoc
    EOS_HConnect,
    EOS_Connect_QueryExternalAccountMappingsOptions,
    EOS_Connect_QueryExternalAccountMappingsCallbackInfo{
        This->EOSConnect,
        &Opts,
        EOS_Connect_QueryExternalAccountMappings,
        [weakThis = GetWeakThis(this), Opts, Batch, OnDone, EOSUser, EntryType]{
            const EOS_Connect_QueryExternalAccountMappingsCallbackInfo *Info {
                if (auto This = PinWeakThis(weakThis))
                {
                    TArray(FResolvedFriendEntry) Results;

                    auto FreeMem = [Opts]{} {
                        EOSString_Connect_ExternalAccountId::FreeFromCharListConst(
                            Opts.ExternalAccountIdCount,
                            Opts.ExternalAccountIds);
                    };

                    if (Info->ResultCode != EOS_EResult::EOS_Success)
                    {
```

C++20 introduces coroutines with `co_await`

- Async/await just like in C#
- No deep nesting
- Compiler guarantees return
- Coroutines spec is very powerful and can do a lot more than just async tasks
- C++20 gives us coroutines spec
- But leaves building a task library to the developer

```
static TTask<void> VoidCoroutine()
{
    co_await FDelayOnAnotherThread{};
    co_return;
}

static TTask<int> SimpleCoroutine()
{
    co_await VoidCoroutine();
    co_return 5;
}

TTask<int> AsyncMain()
{
    int Value = co_await SimpleCoroutine();
    printf("got value from SimpleCoroutine: %d\n", Value);
    co_return 0;
}
```

REDPOINT

leaves task library to developer, since every C++ project is different

Core C++20 coroutines concepts



Let's cover some basic concepts that we need to know so we can design our task library

Two sides of coroutines

Coroutines

Functions that get translated to a state machine:

- ***promise object***
Manipulated from inside the coroutine. This is the thing that specifies how the coroutine will behave.
- ***coroutine handle***
Can be used to get the promise object during suspend/resume.
- ***coroutine state***
Internal dynamically allocated storage.

Awaitables

Things that coroutines can `co_await` on, with the following functions:

- ***await_ready***
Check if `await_suspend` can be skipped (is the result already available?)
- ***await_suspend***
Called to suspend the calling coroutine, with the coroutine handle passed in.
- ***await_resume***
Returns the result of the awaitable.

REDPOINT™

How does the compiler know a function can be a coroutine?

- Either `promise_type` alias/subtype on the returned type:

```
struct task
{
    struct promise_type
    {
        task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

task resuming_on_new_thread(std::jthread& out)
{
```

- Or via `coroutine_traits`:

(This is the variant we will be using for our library)

type parameters for `coroutine_traits` is:

- The return type of the function, followed by
- The type of each function argument

```
// Specialise coroutine_traits so the compiler knows the promise type to use.
namespace std
{
    template <typename ReturnType>
    struct coroutine_traits<Task<ReturnType>>
    {
        using promise_type = ::TCoroutinePromise<ReturnType>;
    };
}
```

REDPOINT

- Our `coroutine_traits` specialization here is for a function that returns `Task<ReturnType>` and accepts **no** arguments
- We will add support for coroutine function arguments later in this talk once we get the basics done (since it interacts with lifetime control)
- For a function to actually **be** a coroutine, it needs to use `co_await`, `co_return`, etc.

Required promise object functions

- ***get_return_object***
Used to construct the actual return value from the coroutine e.g. `TTask<ReturnType>`
- ***initial_suspend***
Called at the start of the coroutine. Can be used to move the coroutine execution to a different thread before anything executes.
- ***final_suspend***
Called at the end of the coroutine.
- ***return_void / return_value***
Called when the `co_return` expression is used, prior to the coroutine being destroyed.
- ***unhandled_exception***
Called on an unhandled exception, `std::current_exception()` can be used to get the exception and store it for rethrowing later.

REDPOINT™

Optional promise object functions

- ***await_transform***

Accepts an awaitable as the first argument, and can return a new awaitable. Useful if you need to intercept `co_await` or wrap awaitables.

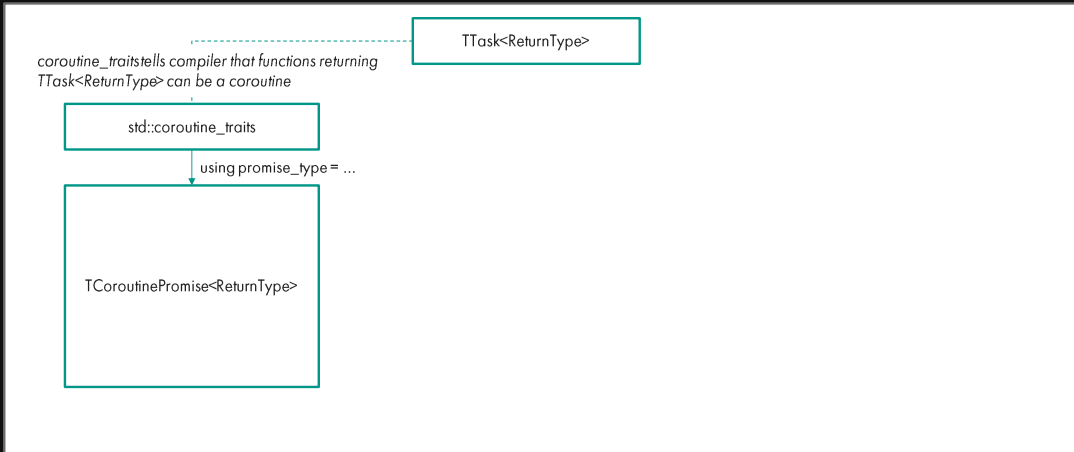
- ***yield_value***

Called when the `co_yield` expression is used. Only useful if you're implementing generators or something like `IAsyncEnumerable`.

Task library design

REDPOINT

Type relationships / Target design



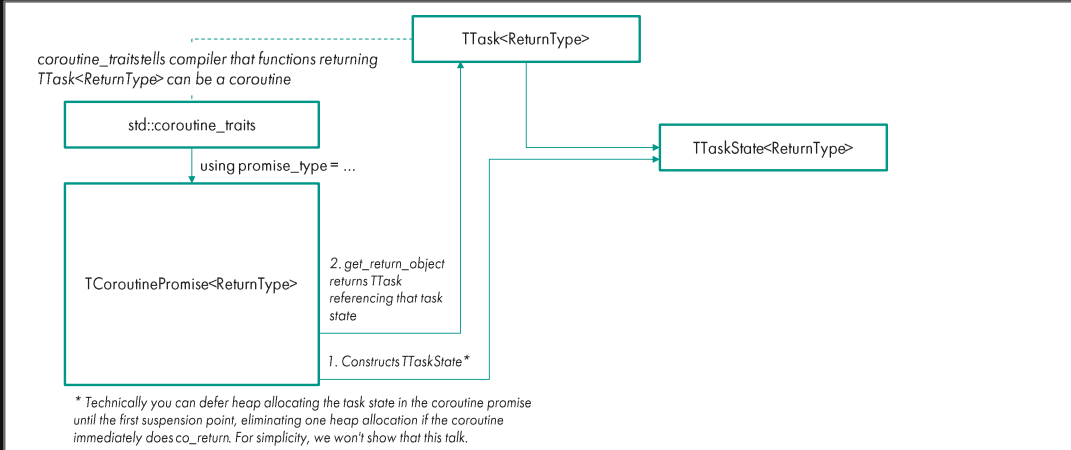
REDPOINT

TTask return type

coroutine_traits to tell compiler that functions returning this type can be coroutines

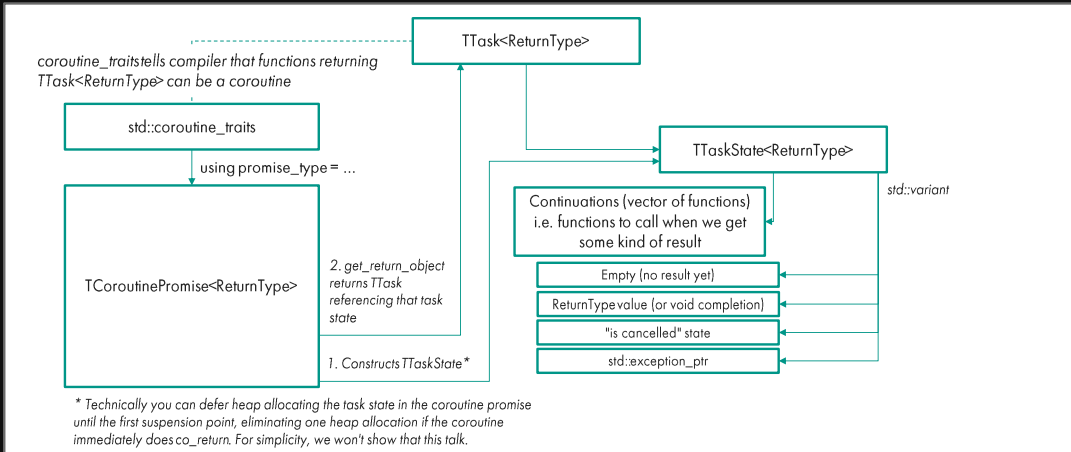
TCoroutinePromise type specifies how the coroutine will behave

Type relationships / Target design



Need some kind of shared state between `TTask` and `TCoroutinePromise`
So that callers with the `TTask` can get notified when the coroutine completes and get the return value

Type relationships / Target design



Finally, our task state needs to contain:

- The list of continuations (functions we will call when we get a result), and
- A variant of either:
 - Empty (no result yet)
 - `ReturnType` value (or void completion token)
 - "is cancelled" token
 - `std::exception_ptr` if you have exceptions enabled and want to be able to rethrow them to the code that is `co_await`'ing

Optimization: Allow `TTask` to have a `std::variant` of a `ReturnType` value or `TTaskState`, skipping the heap allocation for `TTasks` that immediately have a value from e.g. "FromResult" helper

We won't cover that in this talk because we don't have time

Example: 01_SimpleCoroutine, 02_SimpleAwaitable

- These examples are much more simplified than design outlined (we will get there but I want to start as simple as possible)

REDPOINT

01_SimpleCoroutine: Walk through execution flow from main into the coroutine and value propagation through the task state. TTask is not yet awaitable.

02_SimpleAwaitable: Walk through behaviour of co_await with
FDelayOnAnotherThread

Example: 03_TaskAwaitable

- Let's build out more of our design

REDPOINT

03_TaskAwaitable:

- Splitting out code to multiple files
- Task.h, now implements the await_* functions that allow coroutines to co_await them
- (Not implementing FromResult in this demo, so it's only got a shared ptr in it)
- TaskState/CoroutinePromise now has a vector of callbacks instead of a single callback
- Other bits and pieces:
 - AsyncMain/Main/MainDispatch
 - main.cpp is a synchronous function
 - MainDispatch is just a list of functions to run on the main thread
 - DelayOnAnotherThread has been updated to use MainDispatch to return to main thread now
 - AsyncMain is the entry point for demos from now on
- FINALLY: Show AsyncMain and then run demo

Example: 04_TaskVoidAwaitable

- Handling void values
- Can't declare a field of type void
- So we need to specialise some of our templates

REDPOINT

04_TaskVoidAwaitable:

- TTaskState<void> specialization
 - bComplete instead of std::optional Value
- TTask specialization with concepts
 - requires clause on await_* functions
- TCoroutinePromise<void> specialization
 - return_void instead of return_value
- Show contents of AsyncMain
- FINALLY: Show AsyncMain and then run demo

Lifetime control

REDPOINT

What about coroutines on objects?

- So far all our coroutines are static functions
- No 'this' pointer
- Supporting coroutines on objects would be useful
- But how do we make sure that it's valid for a coroutine to resume?
 - i.e. handling an object being free across a suspension point

REDPOINT

Propagate lifetime checks to e.g. `std::enable_shared_from_this`

Things we need to do:

1. `TTaskState` -> `TTaskStateBase` + derived variants with "IsValid" overload
 - `TTaskStateStatic`
 - `TTaskStateSharedFromThis`
2. Add "promise_is_valid" to `TCoroutinePromise` and route to state "IsValid"
 - This is how `await_suspend` will check if it's valid to resume the coroutine
3. Update "await_suspend" to use it
4. Update `std::coroutine_traits` to use derived task state based on function signature

REDPOINT

Or your own shared pointer library, whatever is appropriate for you

Example: 05_SharedFromThisLifetime

REDPOINT

05_SharedFromThisLifetime:

- CoroutineTraits.h specialization based on function signature
 - use of requires to detect type of "this" pointer
 - constructs a different TTaskState
- TaskState.h splitting out classes, addition of IsValid
 - still have our specialization of void for base class
- Task.h await_suspend now calls promise_is_valid on the promise handle before resuming
 - note use of std::terminate; we will replace this with cancellation instead later
- CoroutinePromise.h adding promise_is_valid to propagate to the task state
- FINALLY: run, then run again after uncommenting Object.reset() and see we hit std::terminate
- Adaptable to whatever referencing counting / shared pointer library you are using
- We're just using std::enable_shared_from_this for the demos
- Only thing that you need is the ability to go from a raw pointer to some kind of lifetime check

- TTaskState<void> specialization
 - bComplete instead of std::optional Value
- TTask specialization with concepts
 - requires clause on await_* functions
- TCoroutinePromise<void> specialization
 - return_void instead of return_value
- FINALLY: Show AsyncMain and then run demo

Gotcha: the `await_suspend` of the awaitable is checking the lifetime

- Remember: `await_suspend` is on the awaitable
 - NOT the coroutine who wants to do lifetime checking
- You need to make sure all your `await_suspend` implementations do the lifetime check
- If you are using third party libraries that provide `await_suspend`, be aware they will not do your lifetime checks
- Also: `await_suspend` needs to check the lifetime of the handle you are resuming
 - For `TTask`, NOT the lifetime of it's own `TTaskState`

REDPOINT

- could probably use `await_transform` to wrap awaitables that aren't "lifetime aware" to protect against resuming coroutines without the lifetime check
- `await_transform` prevents the compiler from optimizing in a bunch of cases though, so avoid a generic "catch-all" `await_transform` if possible
- I haven't done this because I don't have any third-party things to `co_await`

Detour: Supporting arguments

- None of our coroutines allow arguments at the moment
- To support this, we need to modify `coroutine_traits` further
- However, now it is ambiguous on the first argument:
 - Is this a member function with a `this` pointer of an object type that isn't supported?
 - Or is it a static function with the first argument a pointer to an object with `enable_shared_from_this` that should not be used for lifetime control?
- `ETaskBinding` enumeration to specialize `TTask`
 - Specializing `TTask` allows us to specialize `coroutine_traits`
 - Because coroutines are inferred from the return type only

REDPOINT

- `ETaskBinding` will tell our task library whether we expect a function to be a member function or static function

Intellisense doesn't handle coroutine_traits properly

- Intellisense gives up with our many specialized coroutine_traits definitions
- You'll want to use a guard on `__INTELLISENSE__` to provide much more simple definitions

REDPOINT

Example: 06_Arguments

REDPOINT

06_Arguments:

- TaskBinding.h, enumeration to indicate whether a function is static or not
 - Unbound enum value we will come back to later
- CoroutineTraits.h
 - COROUTINE_STATIC_ASSERTS: prevent various bad argument types, e.g. const&, pointers, etc.
 - Note coroutine_traits now has ETaskBinding as part of the TTask return type
 - enum propagates to template instantiation of TCoroutinePromise
 - note promise_type takes "Args..." in it's constructor
 - INTELLISENSE rules
- CoroutinePromise.h
 - TaskBinding propagates to get_return_object for TTask (since type must match exactly the return type of the coroutine)
- Task.h
 - TaskBinding is "unspecified" by default (i.e. member function) for the common case
 - Added an implicit cast from TTask<ReturnType, AnyTypeOfBinding> to TTask<ReturnType>

- This makes casting away an `ETaskBinding::Static` possible if you want to e.g. put the task of a static coroutine into a list of tasks
- FINALLY: run demo, uncomment `CoroutineWithPointer` to demonstrate static asserts

Imposed cancellation

REDPOINT

What to do when a coroutine can't resume?

- If the 'this' pointer is no longer valid...
 - `std::terminate` isn't a practical option
 - How can the caller know that they will never get a result?
- Make "cancelled" a possible state for tasks
- Cancellation propagates upwards through `co_await`
 - If you `co_await` a task that cancels, you (as a coroutine) also become cancelled

This is where things start to get somewhat non-trivial

What we need to do:

- TTaskState needs to store its result as a variant (and not optional)
- TTaskState needs a better API for emplacing value or cancellation states, as well as adding continuations
- At the moment we have a separate specialization of TTaskState for void. To avoid code duplication, we're going to move the specialization for void into new templates so we can have one TTaskState declaration.
- Likewise, we also need to reduce code duplication for TCoroutinePromise

REDPOINT

- If you are looking through the upcoming code and finding it hard to keep a mental model of which code is on the coroutine side and which code is on the awaitable side, and the multiple task states / lifetime checks when a coroutine awaits another coroutine, that is normal. This starts to get very complicated to keep track of very quickly, so I would strongly recommend writing a test suite to verify that your task library behaves as expected.
- Keep track of the nuance of when await_suspend, promise resuming, etc is all happening is hard, especially because you have to mentally translate coroutines into the state machines the compiler is actually producing.

Example: 07_ImposedCancellation

- General refactoring
 - CoroutinePromise.h
 - TaskState.h
- Simplifying APIs
 - promise_continue/promise_cancel in TCoroutinePromiseBase
 - Replaces promise_is_valid
 - AddContinuation/AddContinuationFromSuspendingParentCoroutine in TTaskStateBase
 - Replaces public access on the TTaskStateBase continuations

REDPOINT

07_Cancellation:

- CoroutinePromise.h
 - Moved based logic to TCoroutinePromiseBase
 - TCoroutinePromise<ReturnType> for return types, with specialization for void
- TaskState.h
 - FVoidCompletenessNeeded template to convert void to FVoidComplete, otherwise passthrough template argument
 - Use FVoidCompletenessNeeded in TTaskStateBase so we can remove the specialization for void
- promise_continue / promise_cancel in TCoroutinePromiseBase
 - Replaces promise_is_valid
 - Standardizes resuming a coroutine that was co_awaiting
 - This is where we emplace cancellation if our *own* lifetime became invalid after resuming from co_await
- AddContinuationFromSuspendingParentCoroutine in TTaskState
 - Replaces logic inside await_suspend in TTask

- This is where we propagate cancellation to other callers
- e.g. on our own Continuations, we see `blsCancellation`
- If `blsCancellation` is true, we call `promise_cancel` to **force** the caller to cancel even if its lifetime is still valid
 - (instead of calling `promise_continue` which only cancels if the caller's lifetime is invalid)
 - This is because we won't have a result for the caller to receive via `await_resume()`
- FINALLY: demonstrate `AsyncMain` with/without `object.reset()` call, show that program now propagates cancellation out to `main()`
 - Note that `AsyncMain` isn't doing the cancellation
 - it is `SimpleCoroutine` upon the `co_await` of `VoidCoroutine` checking its lifetime in `promise_continue` and cancelling
 - this then forces cancellation on `AsyncMain` via `promise_cancel`

Interacting with synchronous code

Deferred tasks and 'AsCallback'

REDPOINT

Deferred tasks (08_DeferredTasks)

We need to be able to have functions that return TTask which aren't coroutines themselves

- These should be able to return a "deferred" TTask
- That is, a task that we will explicitly set a value or cancellation into at a later point

Important to allow asynchronous code to await on callback-based APIs

REDPOINT

- ForwardDecl.h
 - Add deferred task forward declaration so TTask can use in return type of Deferred() static function
- Task.h
 - Modify Task.h to declare static Deferred() function
 - Make TDeferredTask which exposes set value / set cancellation
 - TDeferredTask because we don't want to allow anyone with a TTask to set its value - the value is expected to be set by a coroutine normally!
 - Implementation of template TTask<>::Deferred()
- DelayOnAnotherThread.h/cpp
 - Show implementation of delay using deferred task instead of awaiter
 - Contrived example - obviously we already have an awaiter implementation, but deferred tasks are more useful when wiring up callback-based APIs
- AsyncMain.cpp
 - Show usage of new DelayOnAnotherThread() function
 - and run

AsCallback (09_AsCallback)

We need to allow callback-based functions to start coroutines and broadcast callbacks when they complete.

AsCallback() function which accepts:

- The TTask to wait for completion
- A lambda to run when the task returns a result
- A lambda to run when the task is cancelled (and no result is available)

Callers of AsCallback can then broadcast their callbacks inside those lambdas as appropriate.

REDPOINT

- AsCallback.h
 - Struct template specialization for std::function<> for void completion
 - AsCallback template function
 - Fairly basic, this is more complex if you want to support threading or if you want to allow tasks to contain immediate result values (i.e. FromResult, without a heap allocated state)
- main.cpp
 - We can now switch to using AsCallback
 - Simplifies our handling of AsyncMain() result and our execution loop
- Note that both of these primitives (Deferred tasks and AsCallback) are fairly trivial to implement
- We've solved a lot of hard problems by this point (with the exception of threading), so building primitives on top is easy again

Requested cancellation

Plus 'AsPotentiallyCancelled'

REDPOINT

Requested cancellation

We might want to be able to request that coroutines are cancelled at their next `co_await` point, even if their lifetime is still valid and they have a result from the `co_await`'d expression

Comparison:

- Imposed cancellation propagates up the stack from a coroutine with an invalid lifetime to the callers.
- Requested cancellation propagates down the stack from a coroutine into the deepest point being awaited.

Requesting cancellation is not immediate; the coroutine cancels at the next resumption point.

REDPOINT

- In C# this is done via cancellation tokens passed in as the last parameter to every function, and requires async functions to explicitly check for cancellation
- We can do better though, because we already have imposed cancellation due to C++'s memory model (compared to C# which is GC)

Example: 10_RequestedCancellation

REDPOINT

10_RequestedCancellation:

- TaskState.h
 - Cancellation request state in a FTaskStateCancellationBase
 - This is because as we propagate cancellation deeper, we don't (or can't) be aware of the ReturnType specialization of the task state that is currently being awaited on
 - "AwaitedOn" tracks what coroutine the current coroutine is awaiting on (to propagate downwards)
 - "CustomCancellationBroadcast" is a list of custom requested cancellation handles (for deferred tasks)
 - RequestCancellation function
 - Go from the current coroutine down the stack, requesting cancellation on each coroutine being awaited on
 - Note: Requesting cancellation is not immediate. We request cancellation and then the task actually cancels at the next resumption point
 - Continue clears out AwaitingOn and CustomCancellationBroadcast because the coroutine is done at that point
 - AddContinuationFromSuspendingParentCoroutine

- Coroutine A awaits coroutine B
- B await_suspend -> B "AddContinuation..." with A as the "SuspendingParentCoroutine"
- This is the point where:
 - B knows A is about to suspend or resume
 - B can check if cancellation has been requested on A
 - B updates A's "AwaitingOn" pointer when it is going to suspend
 - B propagates requested cancellation to itself if A already has cancellation requested
 - This handles situations where RequestCancellation fully runs on the stack reaching A, but A hasn't started awaiting B yet
- AddCustomCancellationHandler
 - Used for deferred tasks so they can listen for cancellation requests
- Task.h
 - RequestCancellation / OnCancellationRequested
 - Simple APIs to access underlying state functions
- Other minor changes: Updated DelayOnAnotherThread to accept duration parameter for our example code
- FINALLY: demonstrate AsyncMain examples 1 and 2

AsPotentiallyCancelled

Need to be able to `co_await` tasks that might be cancelled, and "capture" imposed cancellation to stop it propagating all the way to the root of the async stack.

`AsPotentiallyCancelled` takes a `TTask<ReturnType>` and returns `TTask<std::optional<ReturnType>>`, returning an unset optional if the task was cancelled.

Tip: Users should prefer `AsPotentiallyCancelled` over `AsCallback` if running inside a coroutine, because `AsPotentiallyCancelled` is being `co_await`'d and can propagate requested cancellation (`AsCallback` can not)



Example: 11_AsPotentiallyCancelled



11_AsPotentiallyCancelled:

- Actually pretty easy to build with the primitives we already have
- AsPotentiallyCancelled.h
 - EVoidPotentiallyCancelled/FAsPotentiallyCancelledTypes to handle differences between void and non-void tasks
 - AsPotentiallyCancelled
 - Deferred task that we will set the value into
 - AsyncCallback to listen on the target task and handle value vs. cancellation differently
- FINALLY: demonstrate AsyncMain examples 1, 2 and 3, with and without cancelling for 1 and 2

Async lambdas

REDPOINT

Async lambdas & lifetime control

We'd like to have async support for lambdas

But lambdas with captures pose a tricky problem for lifetime control

Lambdas with captures get transformed into anonymous classes

So the first argument of `coroutine_traits` is the anonymous lambda class, not any 'this' pointer in the capture list

Inferring the lifetime is not possible for async lambdas



Solution: unbound lambdas and "Bind"

Require async lambdas to have a non-constructable "IUnbound" as the first argument, and erase it via a Bind function that specifies the lifetime:

```
auto UnboundAsyncLambda = [](IUnbound) -> TTask<int, ETaskBinding::Unbound> {
    // ...
    co_return 5;
};

// Can't call UnboundAsyncLambda here because we can't make an 'IUnbound' for first argument.
co_await UnboundAsyncLambda(IUnbound());

auto BoundAsyncLambda = Bind(this, UnboundAsyncLambda);

// Now we can call the lambda bound to the lifetime of this.
co_await BoundAsyncLambda();
```

REDPOINT

Example: 12_AsyncLambdas



12_AsyncLambdas:

- TaskState.h
 - New derived TTaskStateLateBound class
 - Allows changing the lifetime after the task state is created
 - Will be used by Bind() to set the coroutine lifetime after getting the initial task object
- CoroutineTraits.h
 - New coroutine_traits specialization for lambdas and ETaskBinding::Unbound
- IUnbound.h
 - Declaration of the IUnbound type
- Bind.h
 - Bind providers
 - Go through comments in this file (fairly straight-forward)
 - Bind() functions which forward to bind providers
- FINALLY: demonstrate AsyncMain examples with lambdas

Compiler gotchas and other considerations

REDPOINT

Exception support

- Clang has a bug where capturing an exception and rethrowing it across coroutines results in a crash because it doesn't generate the correct code to handle `std::exception_ptr`
 - <https://github.com/llvm/llvm-project/issues/182584>
- So #if guard around your exception capture-and-rethrow logic for Clang, even if it would otherwise be enabled for the target

REDPOINT™

this bug probably won't show up in debug builds
and in release builds it might cause random effects -> hard to debug

Notes about memory allocation - coroutine state

- C++20 coroutines are stackless
- A coroutine will allocate its state object via operator new
- There are ways to override operator new from the promise type
- *Unfortunately*, you can't really wire this up to any kind of "async stack"
- Even if we could get some kind of "async stack" from parent context, no way to guarantee the coroutine is the next thing awaited (or even awaited at all).

```
static TTask<void> VoidCoroutine()
{
    co_await FDelayOnAnotherThread{};
    co_return;
}

std::vector<TTask<void>> ExampleListOfTasks;

static TTask<void> ScheduleTask()
{
    TTask<void> Task = VoidCoroutine();
    ExampleListOfTasks.push_back(Task);
    co_return;
}
```

Notes about memory allocation - coroutine state

- Compilers are supposed to be able to move the allocation to the parent coroutine frame when possible: (HALO - Heap Allocation eLision Optimization)

The call to `operator new` can be optimized out (even if custom allocator is used) if

- The lifetime of the coroutine state is strictly nested within the lifetime of the caller, and
- the size of coroutine state is known at the call site.

- In practice, this doesn't often happen ☹
 - MSVC is very bad at it
 - Clang is bad at it - can make it somewhat better by adding `[[clang::coro_await_elidable]]` attribute to your TTask on very new Clang versions

`-std=c++20 -O3 -Rpass-missed=coro`

REDPOINT

`-Rpass-missed=coro` on Clang can tell you when it's not eliding, but not why

HALO can never happen for virtual function calls (because the size of the coroutine state is not known at the call site)

`coro_await_elidable` I have not seen make a performance difference in my benchmarks, so it's likely not doing anything

Notes about memory allocation - task state

- I would like to move task state into coroutine promise to eliminate an additional heap allocation.
 - *Note:* In our actual task library implementation, we already eliminate the task state heap allocation if a coroutine never suspends; we put the value directly into the TTask<> that is yet to return to the caller.
- To do this, you need to make the coroutine promise ref-count itself.
- Work is ongoing... some benchmarks are currently faster, some are currently slower.
- A lot of our TTask usage is with virtual calls for which HALO can never remove the coroutine promise heap allocation, so we are unlikely to ever avoid heap allocations entirely.

REDPOINT

If you need to avoid heap allocations (i.e. you are writing high performance code or loops with lots of iterations), you would design a different coroutine library and likely have to omit some of the features we have to induce HALO.

Some experimental work in example 13 in the examples repository.

Things I don't have time to cover

- Thread policies
 - Automatically making sure a coroutine runs on a desired thread, and returning to the correct thread when the continuation happens
- FExecuteOnCancellation
 - `co_await` to register an async lambda that should run if a coroutine is cancelled before returning
 - Kind of a "finally" block to run for coroutines since cancellation can be imposed from lifetime checks, or requested by an external caller at any time
 - Important if you're implementing transactional code in coroutines

REDPOINT

Thread policies, the short version

- Additional template argument on `TTask<>` that defines thread policy for the coroutine
- Use `initial_suspend` to continue code execution on desired thread if needed instead of always returning `std::suspend_never`
- Update `promise_continue` to return to the desired thread of the caller/`co_await`'er when resuming
- Note that the default thread policy needs to cause coroutines to track what thread they're currently on, e.g.
 - Coroutine A (desires game thread) calls coroutine B (doesn't care what thread) calls coroutine C (desires background thread)
 - When coroutine B resumes, it needs to continue running on the game thread (because that is what it started on), even though it doesn't actually desire a particular thread

FExecuteOnCancellation

- Struct that takes in the async lambda to register in its constructor
- Use `await_transform` on `TCoroutinePromiseBase` to detect `co_await`'s of `FExecuteOnCancellation` and use that to add the async lambda to an

"AsyncCancellationFunctions" list on the state

- Execute AsyncCancellationFunctions in order when promise_cancel runs, with "EmplaceCancellation" happening after all the async lambdas have run in order

Things I don't have time to cover

- Exceptions
 - Necessary if you are compiling with exceptions enabled, otherwise debuggers will not breakpoint where they otherwise would
 - `unhandled_exception()` on the promise object, call `std::current_exception()` and stash the `exception_ptr` in the task state
 - `await_resume` rethrow the exception with `rethrow_exception`

REDPOINT

Exceptions

- Necessary if you are compiling with exceptions enabled, otherwise debuggers will not breakpoint where they otherwise would
- `unhandled_exception()` on the promise object, call `std::current_exception()` and stash the `exception_ptr` in the task state
- `await_resume` rethrow the exception with `rethrow_exception`

Questions?

Important Links

- Slides: junerhodes.au/history
- C++20 coroutines spec: en.cppreference.com/cpp/language/coroutines
- Examples/demo: github.com/hach-que/GCAP2026Examples

Social

- Mastodon: mastodon.social/@hq

REDPOINT™